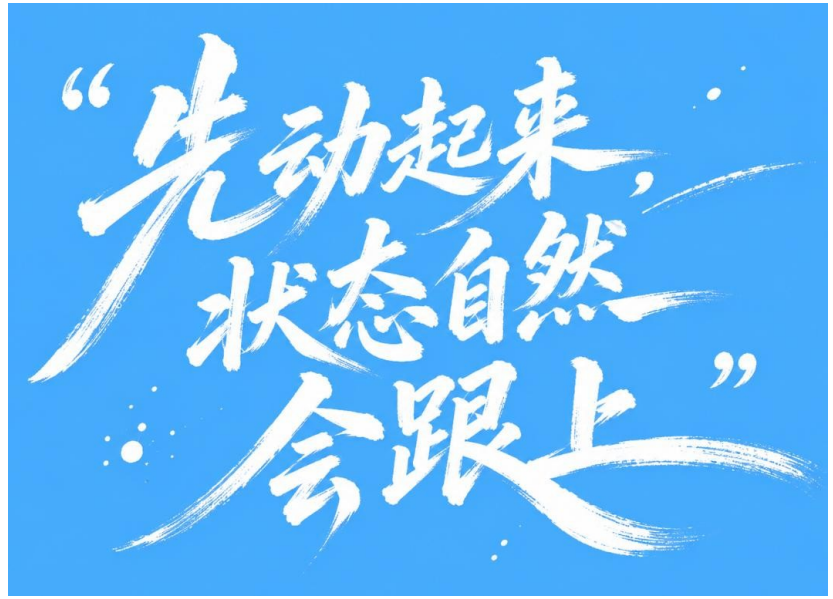
DAY 4

Vibe Coding 理论课

从 LLM 到 Agent - RAG



AI 到底是个什么玩意

主讲：zane

DAY 1
认知与环境

DAY 2
前端 + Skill 机制

DAY 3
后端开发与模型接入

DAY 4
从 LLM 到 Agent - RAG

DAY 5
Git 协作与全栈部署

目录

01	开场与地图	03
02	函数与 AI	09
03	训练与神经网络	32
04	语言数字化	45
05	大语言模型	58
06	微调与部署	72
07	RAG 工程	85
08	Agent 与总结	109

01

开场与地图

1.1	AI 时代，我们先认清这些词	04
1.2	今天的 AI 热词，其实都在同一张地图里	05
1.3	人工智能，是让机器完成原本需要人的智能任务	06
1.4	AI 体系很大，但今天只抓一条主线	07
1.5	我们从头讲一遍：从函数走到大模型	08

1.1 AI 时代，我们先认清这些词

不要被名词吓住：先建立一张能听懂 AI 的地图





1.2 今天的 AI 热词，其实都在同一张地图里

从工具、模型到实体智能

Codex	**Claude Code**	LLM	RAG
Agent	具身智能	token	**openclaw**

这节课不追新闻，而是把这些词放回底层逻辑里。

1.3 人工智能，是让机器完成原本需要人的智能任务

先给一个够用的课堂定义

感知 → 理解 → 生成 → 行动

看见世界，理解含义，组织表达，再调用工具。





ChatGPT Plus



AI 漫剧

1.4 AI 体系很大，但今天只抓一条主线

不用一节课学完整个学科

微积分

概率论

最优化

线性代数

LLM 与 RAG

机器学习

数据处理

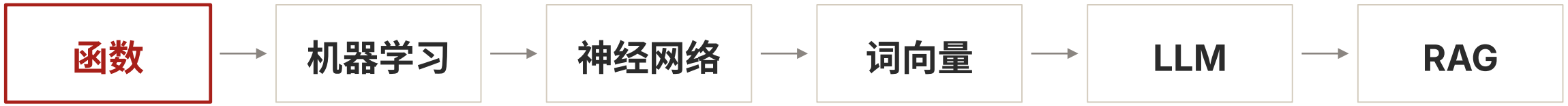
深度学习

工程系统



1.5 我们从头讲一遍：从函数走到大模型

今天的学习路径



先理解“模型在学什么”，再理解“为什么 RAG 能落地”。

02

函数与 AI

2.1	现在，走进 AI 的底层原理	10
2.2	一切先从函数开始	11
2.3	把现实世界符号化，AI 才能处理它	12
2.4	早期人工智能：把人的规则写给机器	13
2.5	问题是，很多规则人类自己也写不清楚	14
2.6	既然不知道函数长什么样，就不硬写了	15
2.7	已知几组 x 和 y，可以反推一条规律	16
2.8	如果一开始看不出规律，人类最常用的方法是猜	17
2.9	先猜一条线：y = wx + b	18
2.10	现实里很少有完美函数，差不多就行	19
2.11	现代 AI 的朴素思路：用数据逼近规律	20
2.12	这条路线叫联结主义	21
2.13	连蒙带猜，居然真的能解决问题	22
2.14	但直线很快就不够用了	23
2.15	激活函数：让线性函数“弯起来”	24
2.16	常见激活函数并不神秘	25
2.17	输入可以有多个，层也可以有很多	26
2.18	足够复杂的网络，可以逼近非常复杂的函数	27
2.19	公式太难看，我们换成神经网络图	28
2.20	中间那层看不见的关系，叫隐藏层	29
2.21	神经元和层数越多，网络就越复杂	30
2.22	第一段小结：AI 底层先抓住这几个词	31

2.1 现在，走进 AI 的底层原理

目标：既浅显又深入，既严肃又有趣

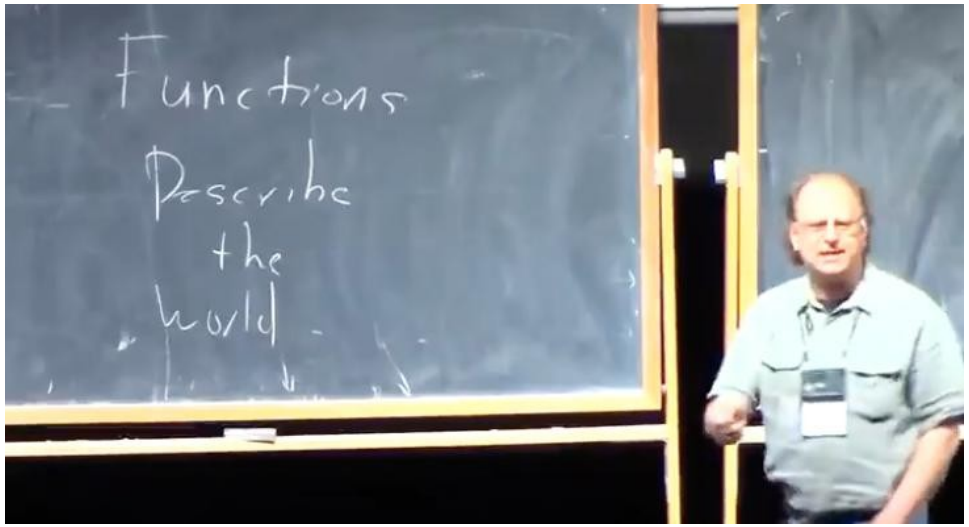
2.2 一切先从函数开始

function describes the world

$$y = f(x)$$

输入 x	规则 f	输出 y

输入 x 经过规则 f，得到输出 y。



美国数据家托马斯*加里蒂

2.3 把现实世界符号化，AI 才能处理它

现实 → 输入 → 规则 → 解释



AI 的第一步，是把问题变成可计算的形式。

2.4 早期人工智能：把人的规则写给机器

符号主义 Symbolic AI



人先想清楚规则，机器负责执行规则。

2.5 问题是，很多规则人类自己也写不清楚

符号主义走到边界





识别小狗函数?

- 耳朵、尾巴、毛色都不稳定
- 规则越写越碎

翻译函数?

- 词典能对应
- 语境和歧义很难写死

所以路线开始转向：让模型从数据里学。

2.6 既然不知道函数长什么样，就不硬写了

从手写规则转向数据学习

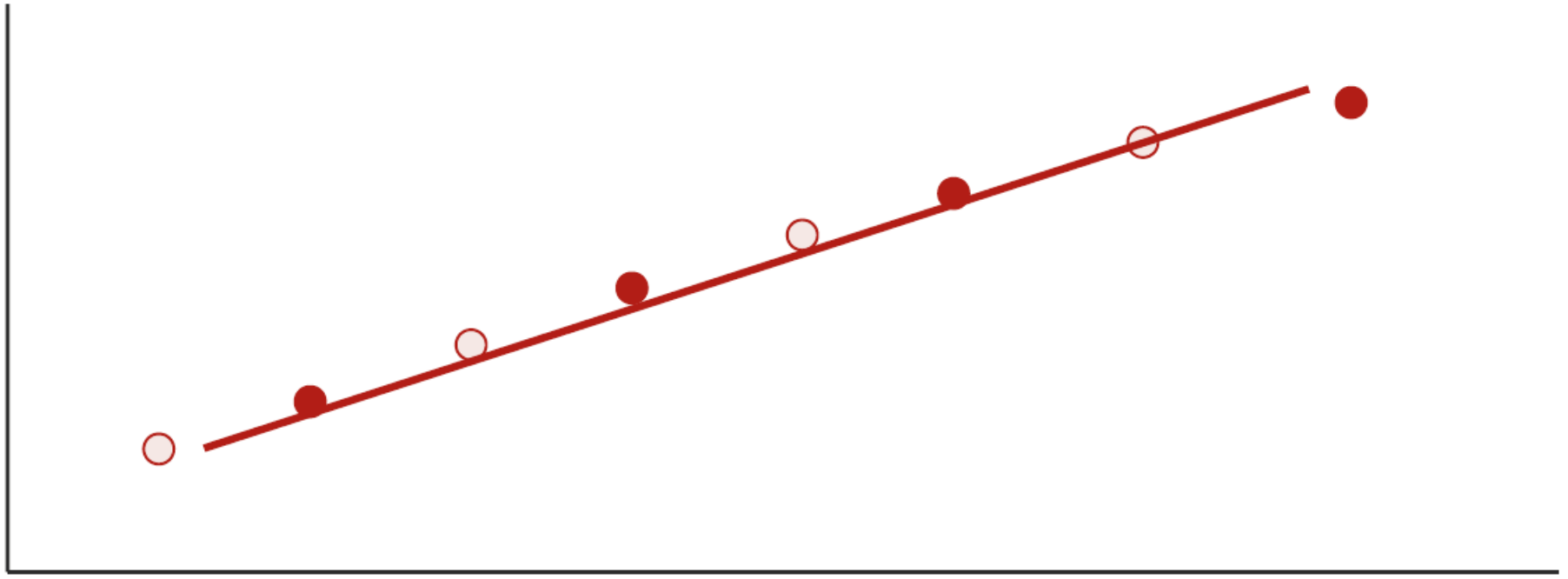


不找现成规则，改成从样本里推规律

这是从符号主义走向现代机器学习的关键转折。

2.7 已知几组 x 和 y，可以反推一条规律

从样本点看见 y=2x

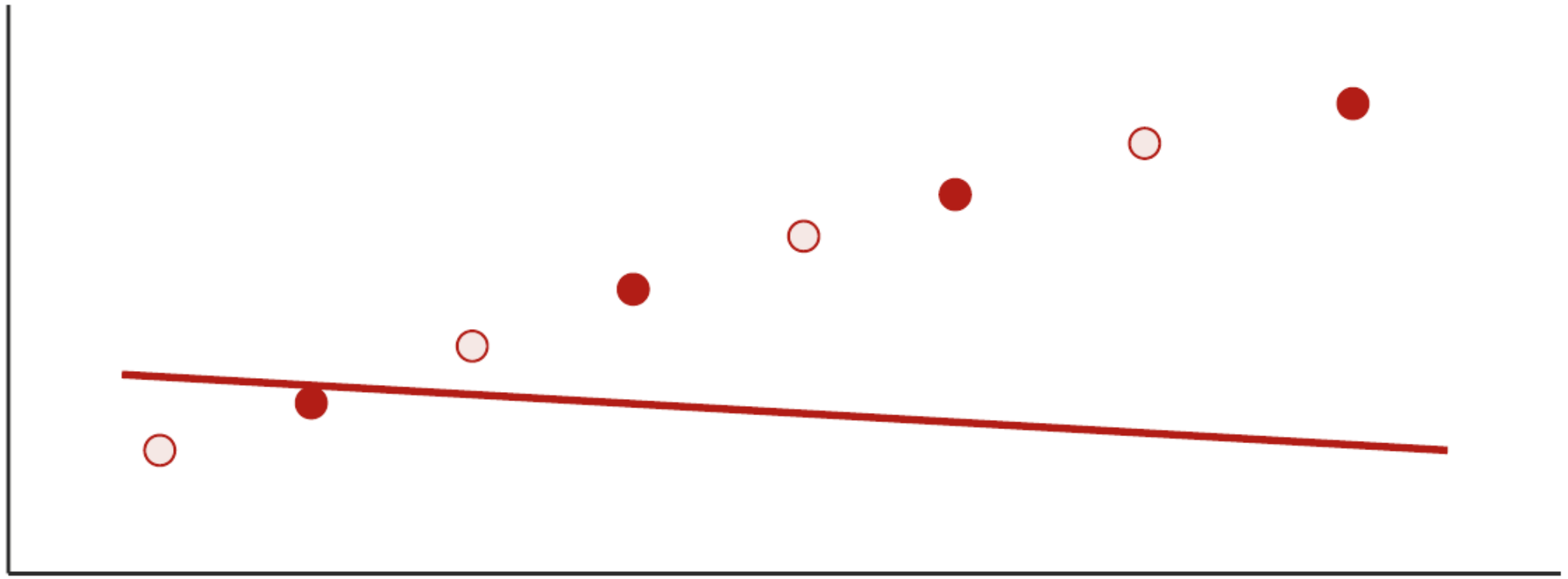


y = 2x

如果规律足够简单，少量样本就能看出来。

2.8 如果一开始看不出规律，人类最常用的方法是猜

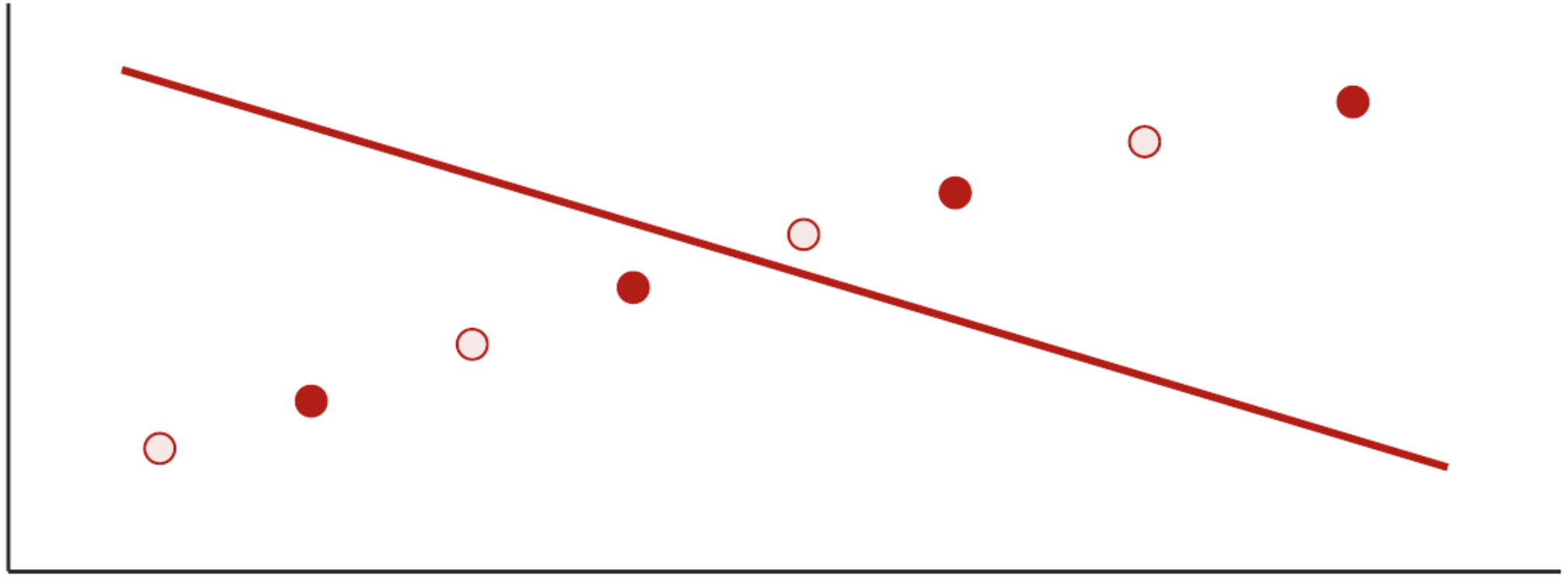
猜一个函数，再看它错多少



机器学习的“猜”，不是乱猜，而是可计算、可更新的猜。

2.9 先猜一条线：y = wx + b

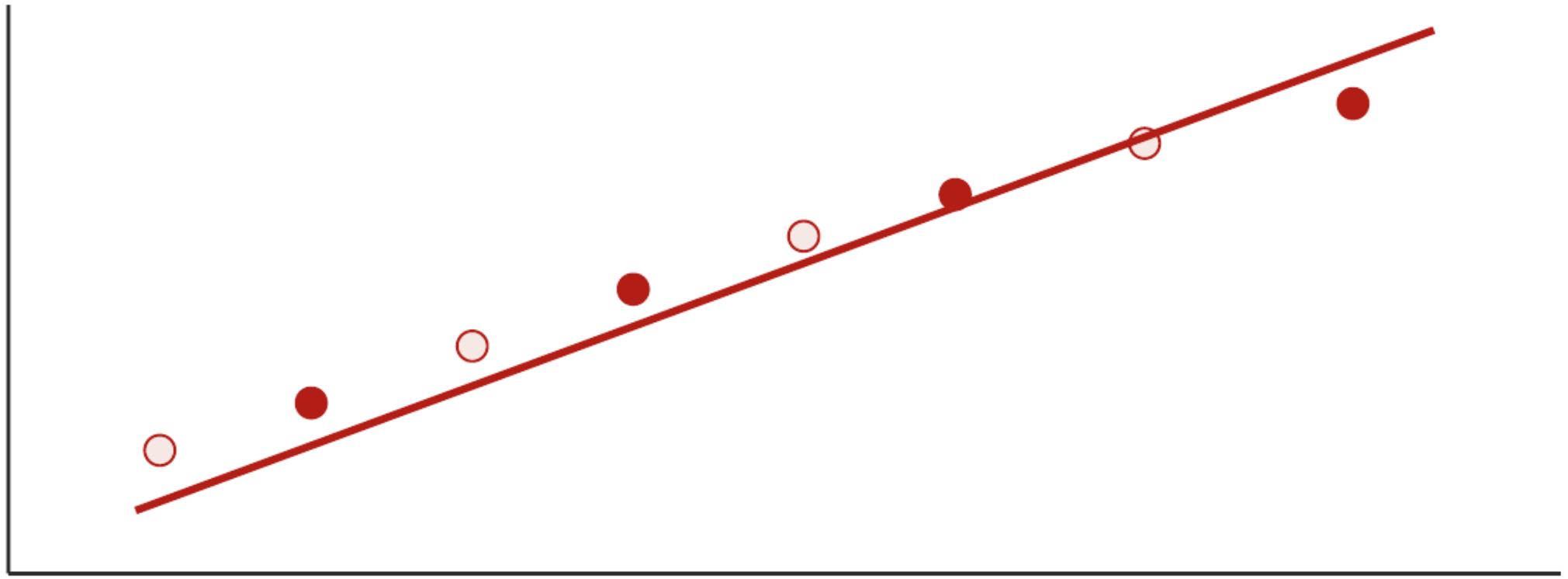
不断调整 w 和 b



参数一变，直线的位置和斜率就会变。

2.10 现实里很少有完美函数，差不多就行

拟合不是追求穿过每一个点



模型要找的是足够接近真实数据的近似关系。

2.11 现代 AI 的朴素思路：用数据逼近规律

猜和简化问题



找不到完美公式，就训练一个足够好的近似函数

这句话后面会一路通到神经网络和大模型。

2.12 这条路线叫联结主义

Connectionism

官方说法

- 用大量简单单元连接成网络
- 通过学习调整连接强度

人话

- 不手写规则
- 让模型从数据中长出规律

神经网络就是联结主义最典型的表达。

2.13 连蒙带猜，居然真的能解决问题

手写数字识别是经典例子

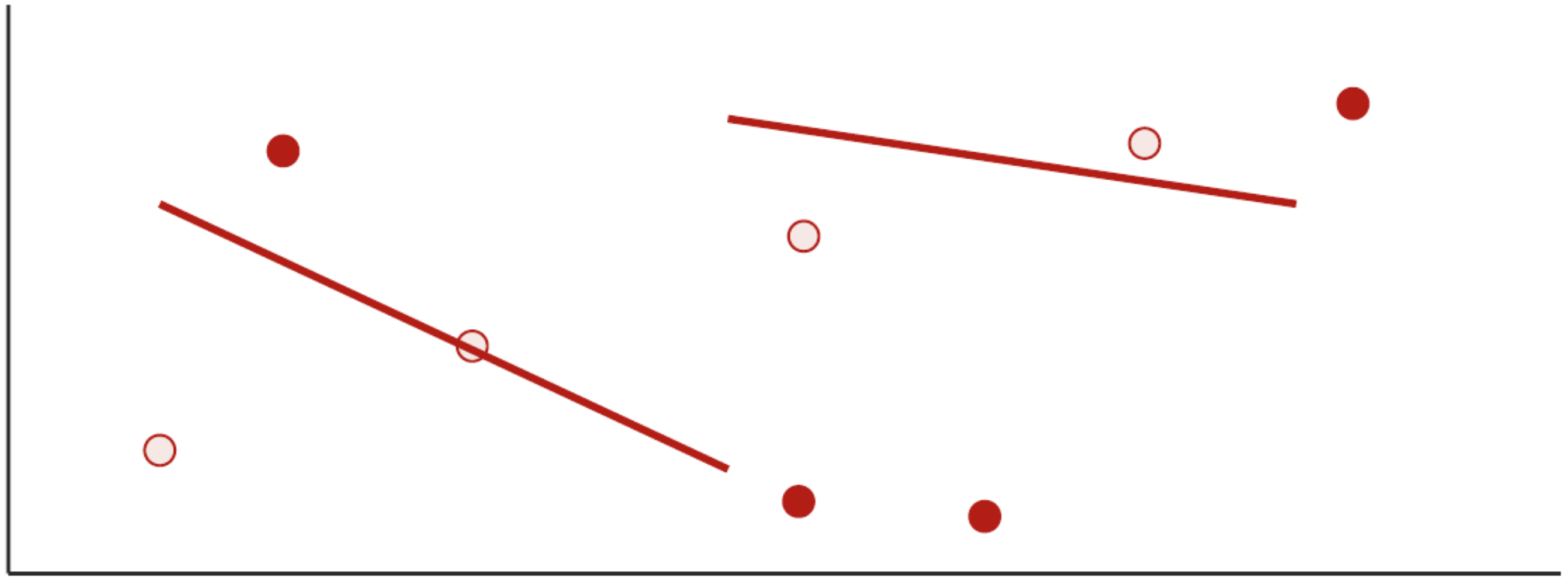
少量参数，也能学会“0 到 9”的视觉差异

https://adamharley.com/nn_vis/cnn/3d.html

有效性被不断验证后，人们才真正重视这条路线。

2.14 但直线很快就不够用了

从线性关系到非线性关系



数据一复杂，线性函数就很难贴近真实分布。

2.15 激活函数：让线性函数“弯起来”

在线性外面套一层非线性

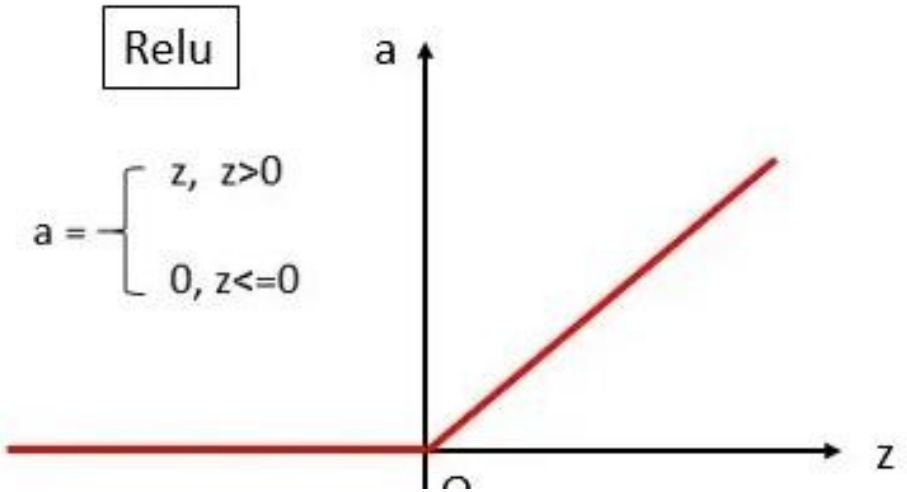
$$f(x) = g(wx + b)$$

线性部分	激活函数	非线性输出

g 可以是 ReLU、sigmoid、tanh，也可以是其他非线性函数。

2.16 常见激活函数并不神秘

名字很数学，作用很直观



ReLU	**Sigmoid**	sin	GELU

它们共同的作用，是让模型表达更复杂的关系。

2.17 输入可以有多个，层也可以有很多

曲线不够弯，就继续套娃

套娃！

$$g(w_3 \cdot g(w_1 x_1 + w_2 x_2 + b_1) + b_2)$$

多输入	多参数	多层



多输入、多参数、多层组合，复杂度就被堆起来了。

2.18 足够复杂的网络，可以逼近非常复杂的函数

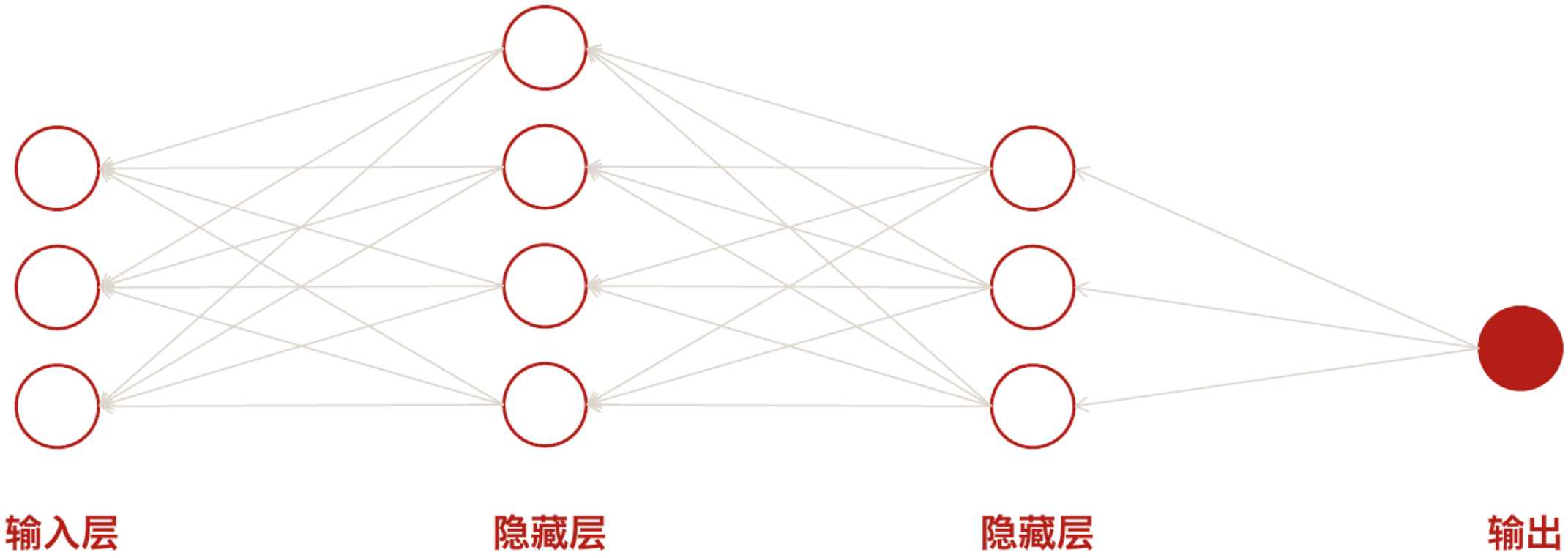
万能逼近的直觉

很多简单变换叠起来，就能描述复杂世界

这不是魔法，是函数组合带来的表达能力。

2.19 公式太难看，我们换成神经网络图

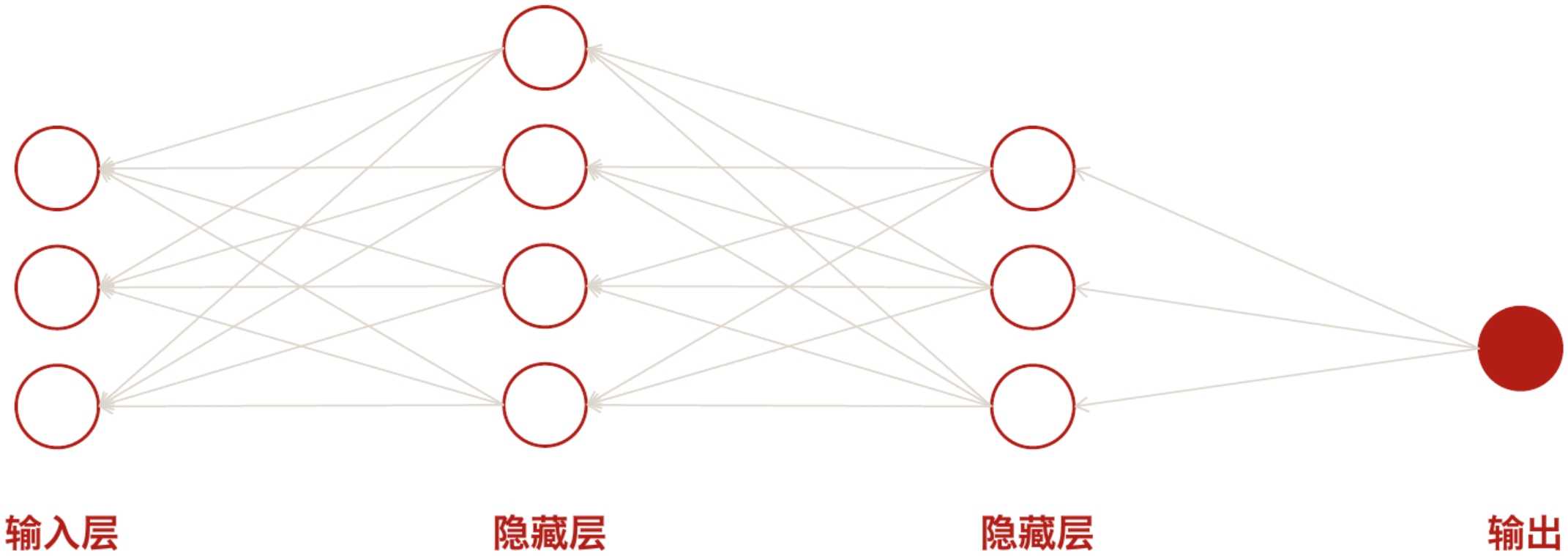
输入层 → 输出层



每个小圆点是神经元，每条连接都带着参数。

2.20 中间那层看不见的关系，叫隐藏层

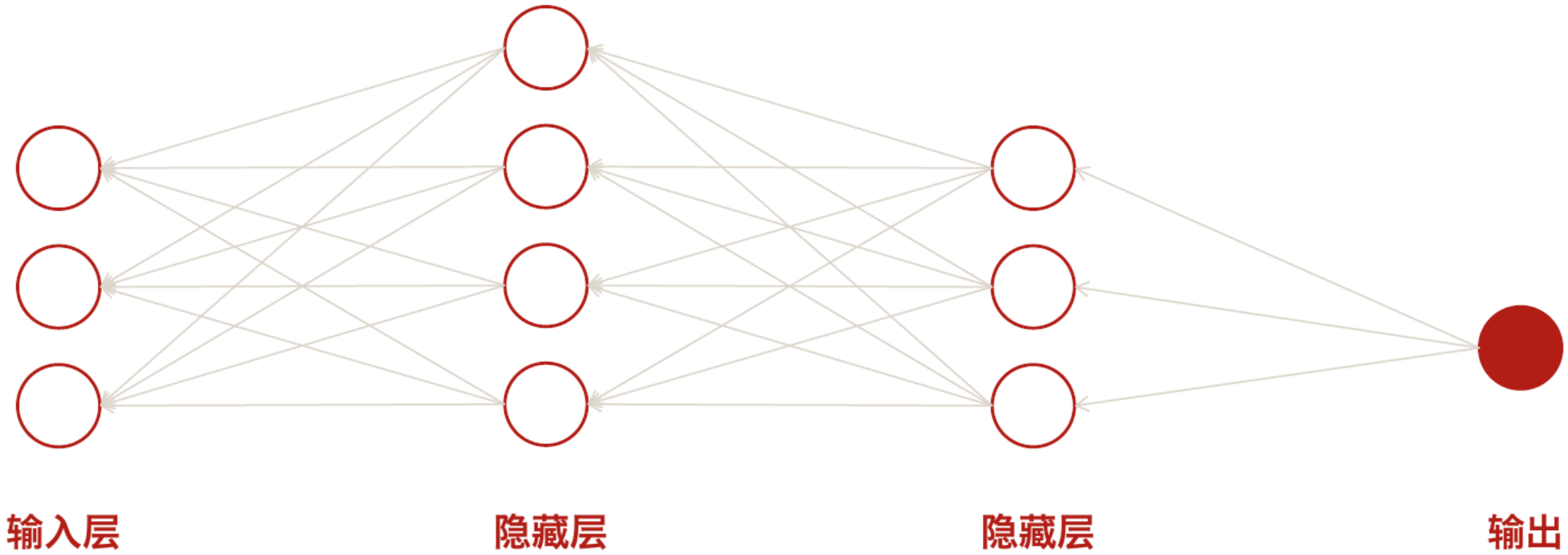
Hidden Layer



隐藏层负责把输入一步步变成更有用的表示。

2.21 神经元和层数越多，网络就越复杂

容量变大，表达能力也变强



复杂网络可以表示更复杂的非线性关系。

2.22 第一段小结：AI 底层先抓住这几个词

函数、规则、数据与网络

函数　符号主义　联结主义　线性

非线性　激活函数　神经网络



后面所有大模型概念，都会回到这些基础直觉。

03

训练与神经网络

3.1	训练神经网络，本质是在找参数	33
3.2	什么样的参数才算好?	34
3.3	拟合得好，和拟合得不好，一眼能看出来	35
3.4	损失函数：把“错多少”变成一个数字	36
3.5	参数怎么求？这就是优化问题	37
3.6	神经网络越大，不代表一定越好	38
3.7	泛化能力：会做新题，才是真的会	39
3.8	机器学习：从数据中学习函数	40
3.9	监督、无监督、半监督，是三种学习方式	41
3.10	机器学习和传统编程，最大的区别在规则来源	42
3.11	深度学习，就是多层神经网络	43
3.12	CNN、RNN 都是神经网络，只是结构不同	44

3.1 训练神经网络，本质是在找参数

w 和 b 是模型真正要学的东西

数据 (x, y) → 参数 (w, b)

样本	训练	参数

有了足够多已知答案，模型就能逐步调出一组可用参数。

3.2 什么样的参数才算好?

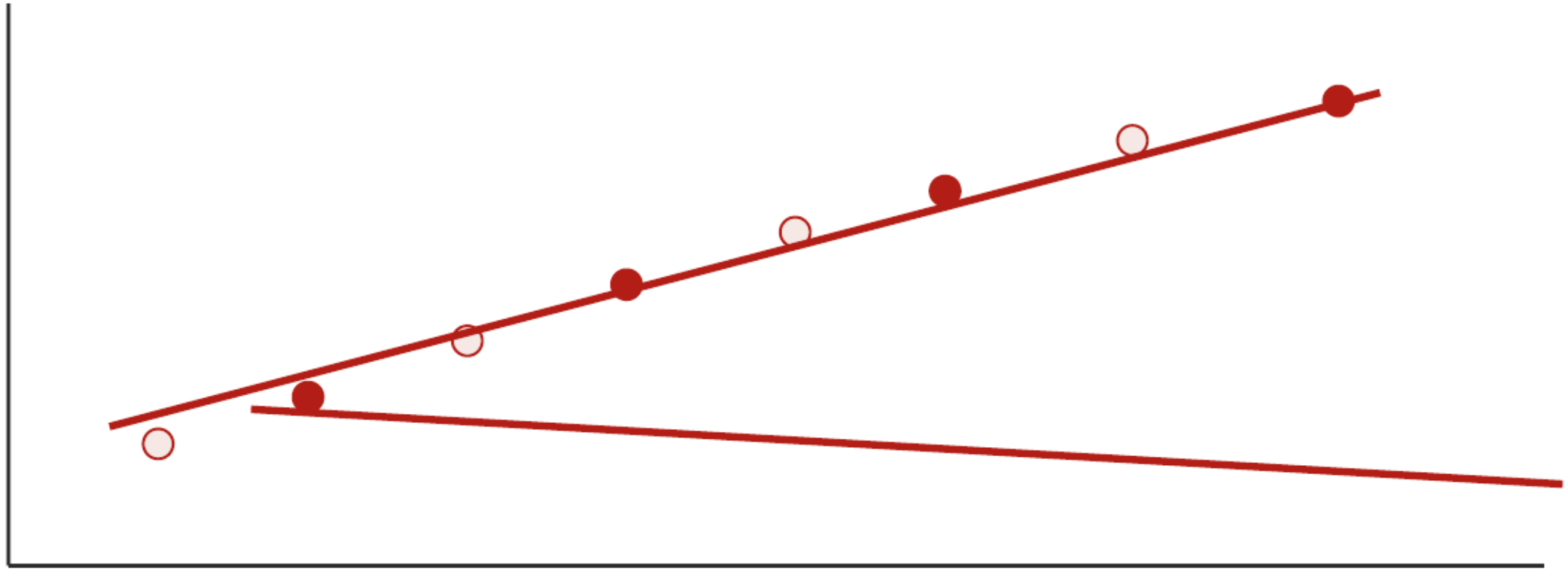
让输出尽可能接近真实答案

好的 w 和 b，会让预测结果更准

模型训练的目标，就是把错误一点点压下去。

3.3 拟合得好，和拟合得不好，一眼能看出来

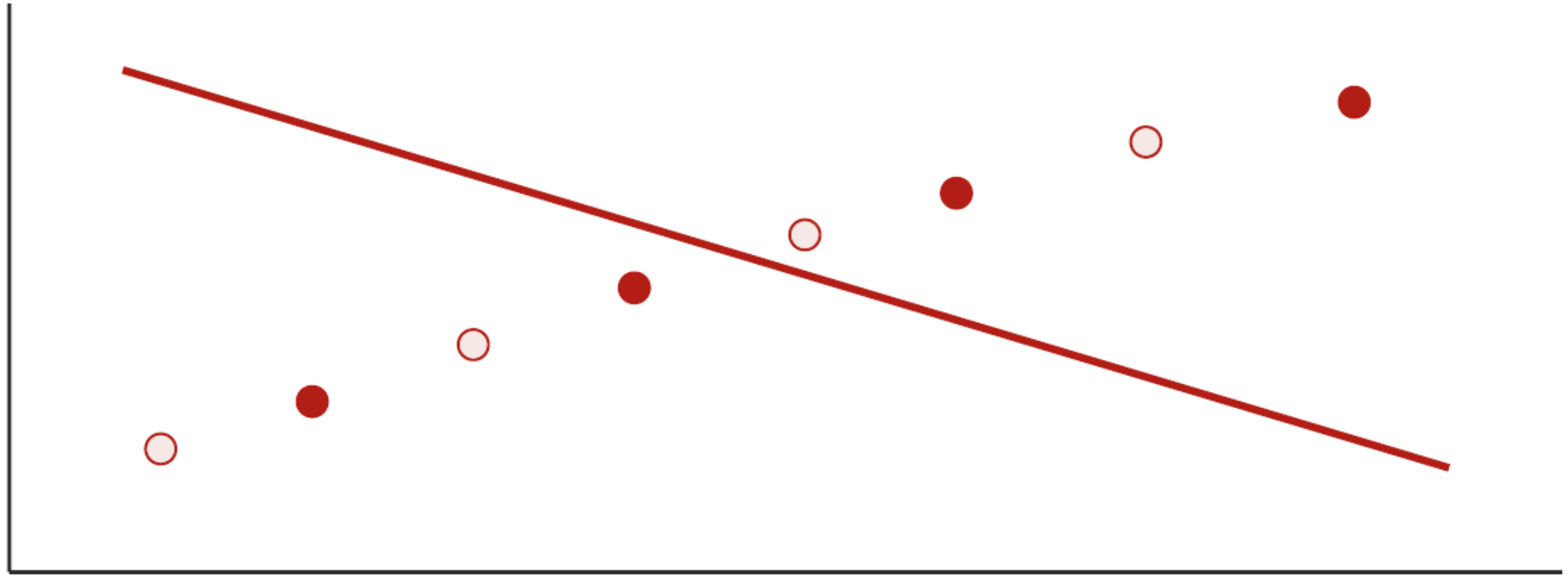
预测线与真实点的距离



图上离得远，数学上就是误差更大。

3.4 损失函数：把“错多少”变成一个数字

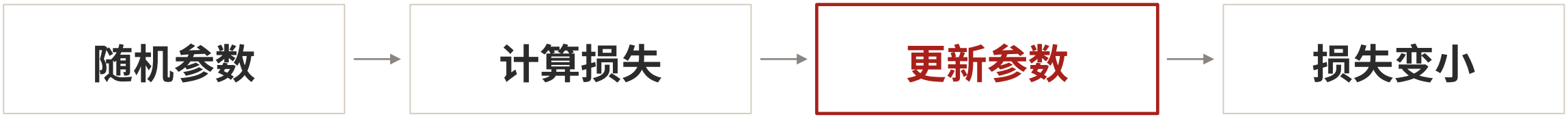
距离越小，损失越小



训练时，模型只需要持续让这个数字变小。

3.5 参数怎么求？这就是优化问题

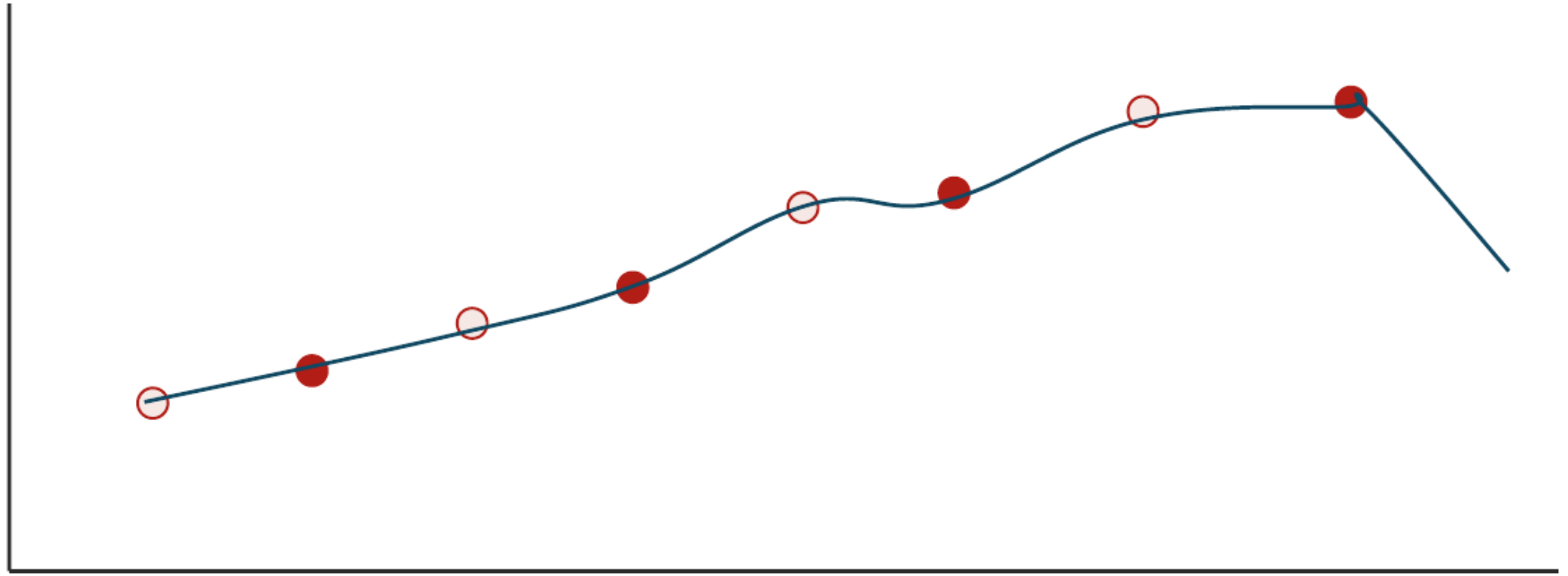
梯度下降只是其中一种方法



复杂神经网络的训练，背后就是大规模数学优化。

3.6 神经网络越大，不代表一定越好

过拟合：把训练题背下来了



模型如果只记住训练数据，就不会真正理解规律。

3.7 泛化能力：会做新题，才是真的会

Generalization

只会背题

- 训练集表现很好
- 换一批题就崩

学到规律

- 训练集表现不错
- 新数据也能处理



AI 最终要追求的是泛化，而不是死记硬背。

3.8 机器学习：从数据中学习函数

不是人写规则，而是数据塑造规则

Data → Model → Prediction

数据	模型	预测

模型就是被数据训练出来的函数。

3.9 监督、无监督、半监督，是三种学习方式

差别在于数据有没有答案

类型	数据特点	典型任务
监督学习	有 x，也有 y	分类 / 回归
无监督学习	只有 x	聚类 / 降维
半监督学习	少量 y + 大量 x	低成本标注

3.10 机器学习和传统编程，最大的区别在规则来源

规则到底是谁写的

传统编程

- 人写规则
- 机器执行规则
- 适合边界清楚的问题

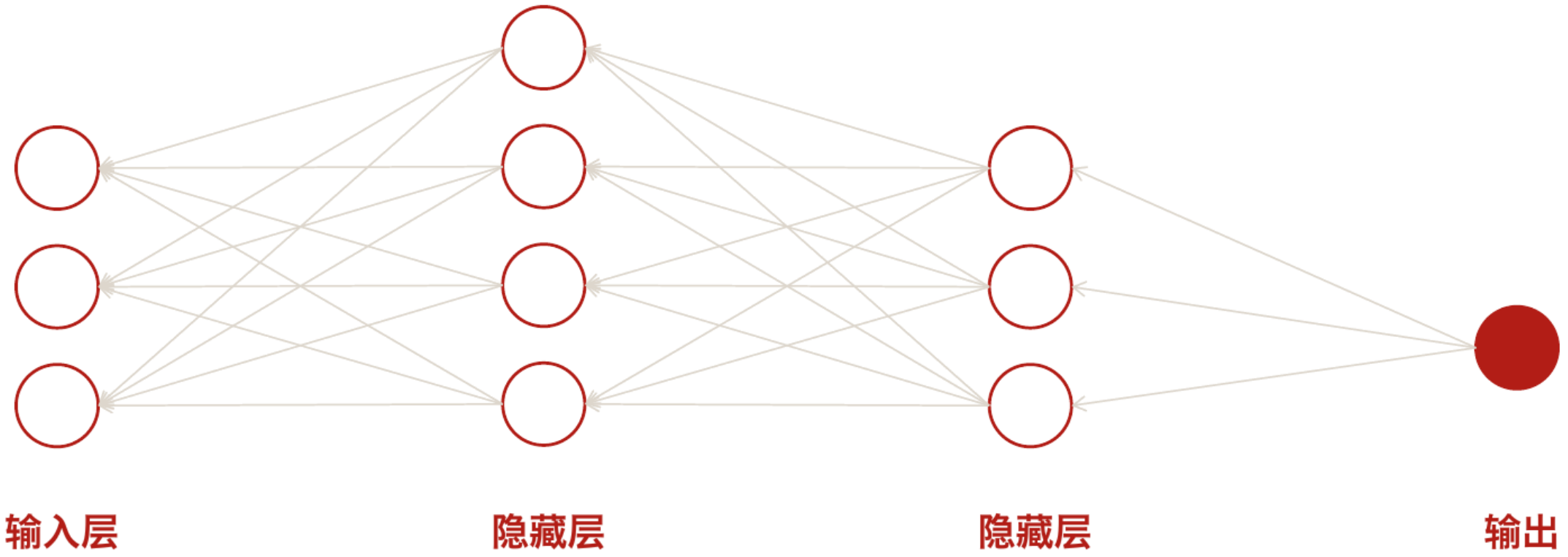
机器学习

- 数据提供规律
- 模型学习参数
- 适合规则难写的问题

这就是 AI 能处理复杂问题的根本原因。

3.11 深度学习，就是多层神经网络

Deep Learning



深度指的是层数更深，表示能力更强。

3.12 CNN、RNN 都是神经网络，只是结构不同

结构服务于任务

CNN	RNN	LSTM	Transformer
图像	**序列**	长序列	文本与多模态

模型架构不是玄学，本质是按任务设计信息流动方式。

04

语言数字化

4.1	从深度学习，走向大语言模型	46
4.2	语言任务，也可以变成神经网络问题	47
4.3	如果让你设计语言模型，你会怎么做?	48
4.4	文字要先变成计算机能识别的数字	49
4.5	方案一：每个词一个编号	50
4.6	方案二：One-hot 编码	51
4.7	更好的方案：Word Embedding	52
4.8	词向量之间的关系，可以用数学计算	53
4.9	词向量会出现一些很神奇的语义现象	54
4.10	所有词向量合起来，就是嵌入矩阵	55
4.11	词向量可以作为神经网络的输入	56
4.12	第二段小结：语言先要数字化	57

4.1 从深度学习，走向大语言模型

语言，也可以被改写成可训练的预测问题

4.2 语言任务，也可以变成神经网络问题

把文字任务写成输入和输出

预测下一个字

- 输入：今天天气真
- 输出：好 / 热 / 冷

判断情感

- 输入：这电影太棒了
- 输出：正向

关键问题变成：如何让计算机理解文字。

4.3 如果让你设计语言模型，你会怎么做?

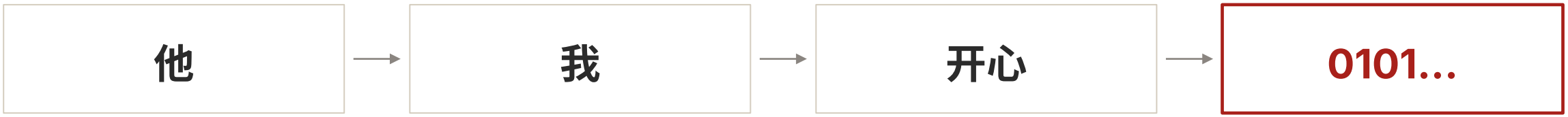
先停一下，想清楚输入是什么

神经网络只能吃数字，不能直接吃文字

所以语言模型的第一步，是把词变成数。

4.4 文字要先变成计算机能识别的数字

编码问题



编码方式不同，会直接影响模型能不能理解词之间的关系。

4.5 方案一：每个词一个编号

简单，但几乎不懂语义

做法

- 他 = 1
- 我 = 2
- 开心 = 3

问题

- 数字大小没有语义
- 词与词的关系丢失

编号能表示身份，但不能表示含义。

4.6 方案二：One-hot 编码

每个词一个超长向量

优点

- 清楚表示词身份
- 实现简单

缺点

- 维度极高
- 向量彼此正交
- 语义关系仍然没有

词表越大，向量越长，问题越明显。

4.7 更好的方案：Word Embedding

用合适维度表示词义

桌子 = (0.12, 0.00, 0.43, 0.50, ...)

词	向量	语义

每个词变成一个稠密向量，既不太短，也不太长。

4.8 词向量之间的关系，可以用数学计算

余弦相似度 / 点积

$$similarity(a, b) = \cos(\theta)$$

向量 a	夹角 θ	向量 b

语义相近的词，在向量空间里会更接近。

4.9 词向量会出现一些很神奇的语义现象

语义关系被编码进空间



比如“桌子-椅子”和“鼠标-键盘”可能呈现相似方向。

4.10 所有词向量合起来，就是嵌入矩阵

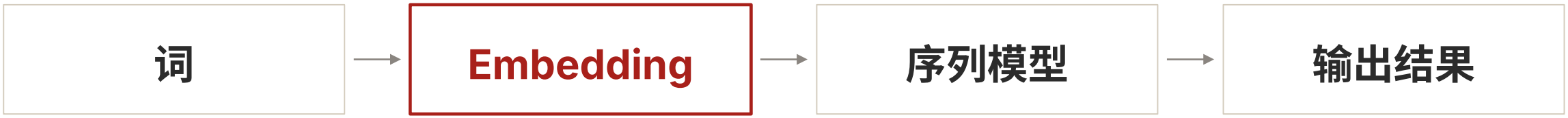
Embedding Matrix

词	d1	d2	d3
桌子	0.12	0.00	0.43
椅子	0.10	0.03	0.39
键盘	0.61	0.28	0.14

嵌入矩阵本身就是训练出来的模型组件。

4.11 词向量可以作为神经网络的输入

RNN、LSTM 曾经是处理序列的经典模型



从这一刻开始，自然语言就进入了神经网络世界。

4.12 第二段小结：语言先要数字化

编码、词向量、嵌入矩阵

编码	One-hot	Embedding
余弦相似度	嵌入矩阵	序列模型

大语言模型的输入，本质上也是这些向量。

05

大语言模型

5.1	终于，来到大语言模型	59
5.2	LLM 为什么叫“大”？因为参数很多	60
5.3	大模型学说话的核心任务：预测下一个词	61
5.4	它不是在查事实，而是在算概率	62
5.5	Transformer 的关键，是注意力机制	63
5.6	预训练：把模型扔进海量文本里学习规律	64
5.7	对齐：教模型按人类希望的方式回答	65
5.8	参数、数据和算力一起变大，能力会发生跃迁	66
5.9	但模型越大，训练成本越夸张	67
5.10	闭源模型：能力强，但核心不可控	68
5.11	开源模型：可部署、可定制，但有门槛	69
5.12	理解主流模型，不要只问谁最强	70
5.13	普通人怎么选模型？先看四个维度	71

5.1 终于，来到大语言模型

LLM 是一个极大的、以语言预测为核心的神经网络

5.2 LLM 为什么叫“大”？因为参数很多

Large Language Model

参数就是前面讲过的 w 和 b

参数越多，模型能表达的关系通常越复杂，但成本也越高。

5.3 大模型学说话的核心任务：预测下一个词

Next Token Prediction

今天天气真 → 好

前文	概率	下一个词

一个简单任务，重复亿万次，就学到大量语言规律。

5.4 它不是在查事实，而是在算概率

不要把生成误认为思考

模型生成的是“最像答案”的文字

这句话非常重要，后面理解幻觉和 RAG 都要用到。

5.5 Transformer 的关键，是注意力机制

Attention lets tokens look at each other

看全句

找关系

并行训练

Attention

长文本

上下文

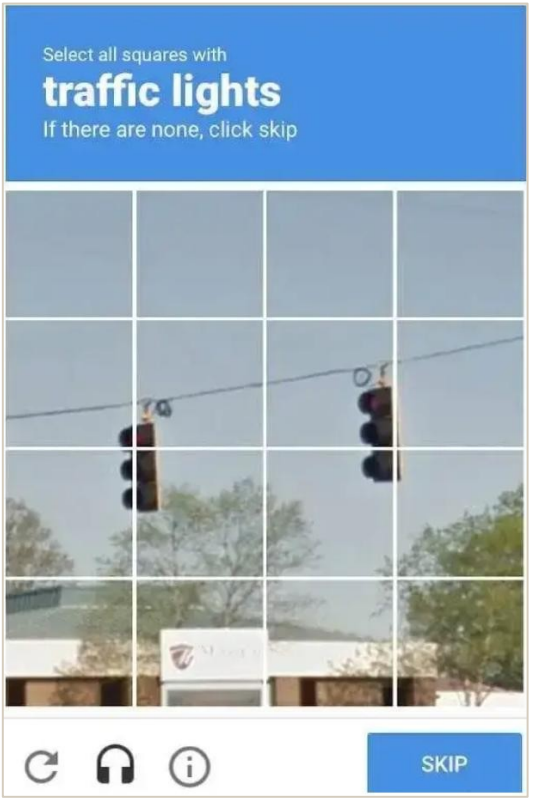
权重分配

模型读句子时，会自动判断哪些词更值得关注。

5.6 预训练：把模型扔进海量文本里学习规律

Pre-training

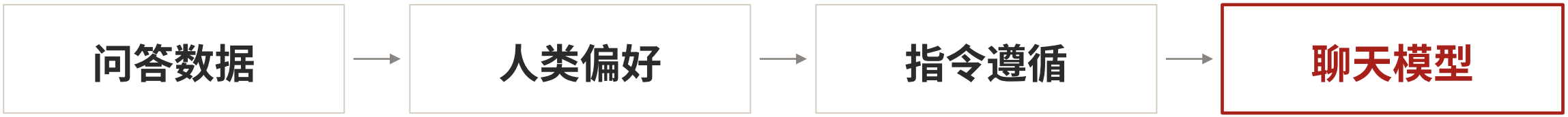
互联网文本 → 预测下一个词 → 学习语言规律 → **基座模型**



基座模型像一个知识很多但还不会好好聊天的学霸。

5.7 对齐：教模型按人类希望的方式回答

Alignment



对齐之后，模型才更像我们日常使用的 ChatGPT 类产品。

5.8 参数、数据和算力一起变大，能力会发生跃迁

规模带来涌现能力

量变积累到一定程度，会出现复杂能力

推理、翻译、写代码等能力，都是在规模化训练中逐渐显现的。

5.9 但模型越大，训练成本越夸张

大模型不是普通团队随便训练的

收益

- 能力更强
- 泛化更好
- 适配任务更多

代价

- 算力昂贵
- 数据昂贵
- 工程复杂

所以真正从零训练大模型，通常只有少数机构能承担。

5.10 闭源模型：能力强，但核心不可控

API 调用为主

优点

- 效果稳定
- 接入简单
- 安全策略成熟

缺点

- 持续付费
- 数据出域风险
- 难以深度定制

典型使用方式是通过 API 调用。

5.11 开源模型：可部署、可定制，但有门槛

权重可下载，工程要自己做

优点

- 成本可控
- 数据不出域
- 可改可部署

缺点

- 部署复杂
- 硬件要求高
- 效果需要调优

开源模型适合对隐私、成本和控制权更敏感的场景。

5.12 理解主流模型，不要只问谁最强

要按场景选模型

场景	优先关注	选择逻辑
写代码	推理与工具调用	看代码质量和稳定性
长文档	上下文长度	看长文本能力
企业落地	隐私与成本	看部署和合规
多模态	图像/语音能力	看输入输出形态

没有绝对最好的模型，只有最适合当前任务的模型。

5.13 普通人怎么选模型？先看四个维度

效果、成本、隐私、部署

效果	成本	隐私	部署

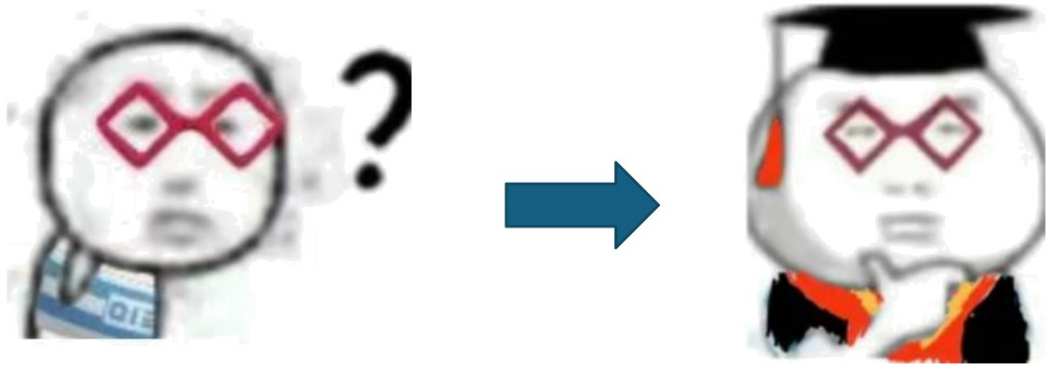
不差钱追求效果，用 API；重视私有化和成本，再考虑开源部署。

06

微调与部署

6.1	通才模型，如何变成专才模型?	73
6.2	微调：把通才培养成专才	74
6.3	预训练像盖毛坯房，微调像做精装修	75
6.4	微调一般在调什么?	76
6.5	微调也有成本，不是免费的魔法	77
6.6	微调之后，模型到底在哪里?	78
6.7	本地部署：让模型跑在自己的机器上	79
6.8	本地部署的好处很明确	80
6.9	本地部署最大的门槛，是硬件	81
6.10	快速估算：参数量约等于显存压力	82
6.11	消费级显卡和生产级显卡，定位不同	83
6.12	本地部署小结：不是越早越好	84

6.1 通才模型，如何变成专才模型?



微调与本地部署，是大模型落地的重要分岔路

6.2 微调：把通才培养成专才

Fine-tuning

基座模型

- 什么都懂一点
- 通用能力强
- 不懂你的格式

微调模型

- 更懂领域
- 更守格式
- 更适合固定任务

微调不是从零训练，而是在已有模型上继续训练。

6.3 预训练像盖毛坯房，微调像做精装修

成本差距非常大

预训练

- 从零开始
- 海量数据
- 极高算力

微调

- 基于已有模型
- 垂直数据
- 成本低很多

微调的核心是“小范围改变参数”，让模型更适合你的任务。

6.4 微调一般在调什么?

SFT 与领域微调最常见

SFT	领域微调	格式微调	风格微调
指令微调	法律/医学/金融	固定输出	组织口径

微调会让模型“偏科”，而这个偏科正是业务需要。

6.5 微调也有成本，不是免费的魔法

数据、算力、维护

投入

- 高质量标注数据
- 训练与评测算力
- 持续版本维护

适合

- 需求明确
- 数据充分
- 长期稳定

如果需求经常变，RAG 往往比微调更灵活。

6.6 微调之后，模型到底在哪里?

云端托管 or 自己持有

API 微调	→	**厂商托管**
开源微调	→	**自己部署**

用 API 微调不等于模型就在你自己手里。

6.7 本地部署：让模型跑在自己的机器上

Local Deployment

数据不出你的服务器，调用也不依赖第三方

政府、金融、军工等场景尤其重视这一点。

6.8 本地部署的好处很明确

安全、成本、控制权

数据安全

长期成本可控

系统完全可控

可接内部系统

它适合调用量大、隐私要求高、需要深度定制的组织。

6.9 本地部署最大的门槛，是硬件

参数要占显存

模型越大

- 参数越多
- 显存越高
- 推理越慢

硬件越贵

- 多卡服务器
- A100 / H100
- 运维成本

第一道问题很现实：你有多少预算买显卡？

6.10 快速估算：参数量约等于显存压力

粗略记住这个量级

模型规模	半精度显存	直观理解
7B	约 14GB	单张中高端卡
13B	约 26GB	更吃显存
70B	约 140GB	多张专业卡

实际部署还会受量化、上下文长度和并发影响。

6.11 消费级显卡和生产级显卡，定位不同

不是只看显存数字

消费级显卡

- 便宜些
- 适合学习实验
- 稳定性和扩展有限

生产级显卡

- 显存更大
- 适合长时间运行
- 成本高

学生阶段可以先从小模型和 API 入手。

6.12 本地部署小结：不是越早越好

按场景决策

适合

- 数据安全要求高
- 调用量巨大
- 需要深度定制

不适合

- 刚起步小团队
- 预算有限
- 需求还不稳定

多数项目初期用 API 更轻，跑通业务后再考虑部署。

07

RAG 工程

7.1	再强的模型，也会幻觉	86
7.2	减少幻觉的关键：让模型现查资料	87
7.3	RAG：检索增强生成	88
7.4	RAG 的常见架构：应用夹在资料和模型之间	89
7.5	阶段一：准备资料	90
7.6	阶段二：回答问题	91
7.7	RAG 解决了大模型落地的几个关键问题	92
7.8	RAG 的一句话总结	93
7.9	标准 RAG 流程可以拆成五步	94
7.10	向量数据库：存语义向量的地方	95
7.11	QA 改写：先把问题改得更适合检索	96
7.12	BM25：关键词检索的经典方法	97
7.13	重排序：把召回结果重新排一遍	98
7.14	Chunk：RAG 里的最小知识片段	99
7.15	Chunk 策略：按长度、标题或语义切分	100
7.16	文件处理，是 RAG 工程的地基	101
7.17	Markdown 很适合做知识库原材料	102
7.18	Markdown 常见语法，只需要掌握几类	103
7.19	常见 Markdown 工具，可以帮助整理资料	104
7.20	docx 处理：不只是提取正文	105
7.21	PDF 要先分清：文本型还是图片型	106
7.22	OCR 模型：把图片里的文字识别出来	107
7.23	现场实操：调用 OCR 模型处理文件	108

7.1 再强的模型，也会幻觉

RAG 的出现，就是为了解决“模型不知道还硬答”的问题

7.2 减少幻觉的关键：让模型现查资料

不要靠重新训练解决一切

既然记不住，就回答前先检索

重新训练太贵，现查资料更灵活、更工程化。

7.3 RAG：检索增强生成

Retrieval-Augmented Generation

先查资料 + 再生成答案

Retrieval	Augmented	Generation

像开卷考试：不是硬背，而是带着参考资料作答。

7.4 RAG 的常见架构：应用夹在资料和模型之间

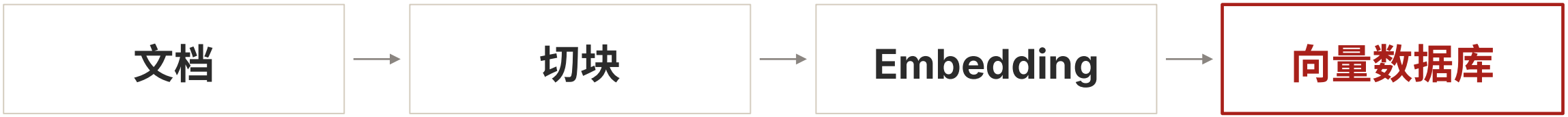
用户问题先经过检索系统



模型不再凭空回答，而是基于检索到的资料回答。

7.5 阶段一：准备资料

文档入库 Pipeline



资料先被切成小段，再转成向量，最后存入向量库。

7.6 阶段二：回答问题

查询 Pipeline



问题和资料都在同一个向量空间里比较相似度。

7.7 RAG 解决了大模型落地的几个关键问题

更新、私有、可追溯

知识可更新

私有数据可用

幻觉显著缓解

答案可引用

注意：RAG 不能彻底消灭幻觉，但能显著降低风险。

7.8 RAG 的一句话总结

先查资料，再回答问题

模型负责表达，检索负责找依据

把“会说话”的模型，接到“有资料”的系统上。

7.9 标准 RAG 流程可以拆成五步

从文档到答案



后面的工程细节，都是在优化这五步。

7.10 向量数据库：存语义向量的地方

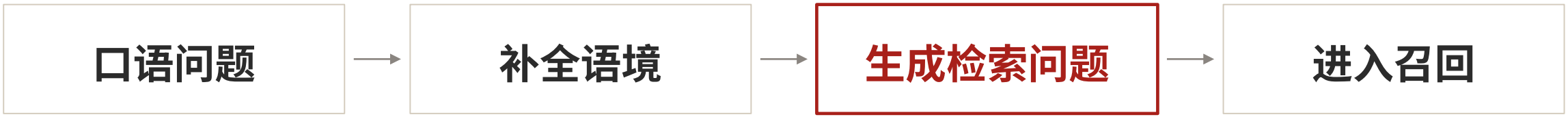
Vector Database

语义相近的文本，向量距离更近

它让系统可以用“意思相近”而不只是“关键词相同”来找资料。

7.11 QA 改写：先把问题改得更适合检索

Query Rewriting



用户怎么问，不一定就是最适合搜索的表达。

7.12 BM25：关键词检索的经典方法

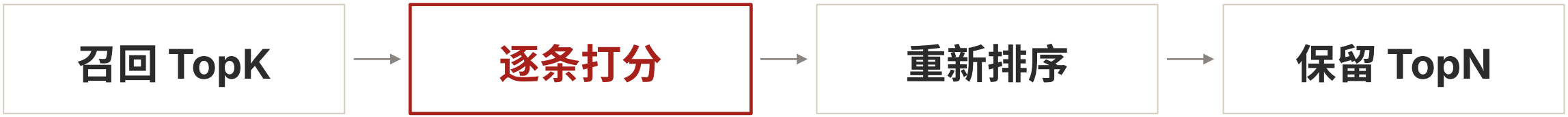
不要只迷信向量检索

关键词匹配，有时比语义检索更稳

尤其是专有名词、编号、法规条款、错误代码等场景。

7.13 重排序：把召回结果重新排一遍

Rerank



召回负责“多找点”，重排负责“挑准点”。

7.14 Chunk：RAG 里的最小知识片段

长文档必须切小块

Chunk 太大，检索不准；太小，语义不完整

切块策略直接决定 RAG 的上限。

7.15 Chunk 策略：按长度、标题或语义切分

没有一种切法适合所有文档

简单切分

- 固定长度
- 加 overlap
- 实现容易

结构切分

- 按标题段落
- 保留层级
- 更适合知识库

好的切块不是切碎，而是保留可检索的意义单元。

7.16 文件处理，是 RAG 工程的地基

Garbage in, garbage out



文档处理不干净，后面的向量库和大模型都会被拖累。

7.17 Markdown 很适合做知识库原材料

结构清楚，层级天然存在

标题 → 段落 → 列表 → 表格

它比杂乱的 Word/PDF 更容易被稳定解析。

7.18 Markdown 常见语法，只需要掌握几类

标题、列表、代码块、表格

`# 标题` `- 列表` ```` ```代码``` ```` `| 表格 |` `> 引用`

这些结构信息，会帮助 RAG 更好地切分和检索。

7.19 常见 Markdown 工具，可以帮助整理资料

先把知识整理成干净文本

Typora	**语雀**	飞书文档	Obsidian

工具不是重点，重点是输出结构清楚、可解析的文档。

7.20 docx 处理：不只是提取正文

标题、表格、图片都要考虑

docx	正文	标题层级	表格	图片说明

如果表格和图片丢了，知识库就会缺关键信息。

7.21 PDF 要先分清：文本型还是图片型

两种 PDF 的处理完全不同

文本型 PDF

- 文字可复制
- 可直接解析
- 结构可能混乱

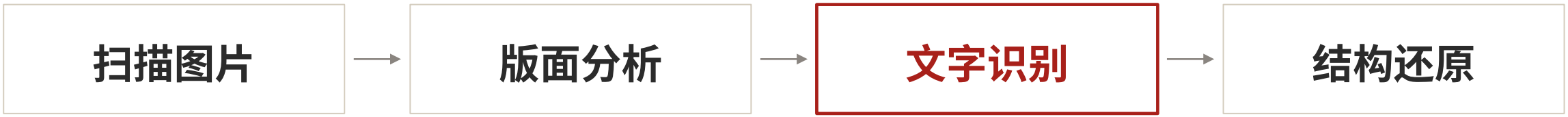
图片型 PDF

- 本质是图片
- 需要 OCR
- 识别质量决定效果

很多扫描版资料，必须先过 OCR。

7.22 OCR 模型：把图片里的文字识别出来

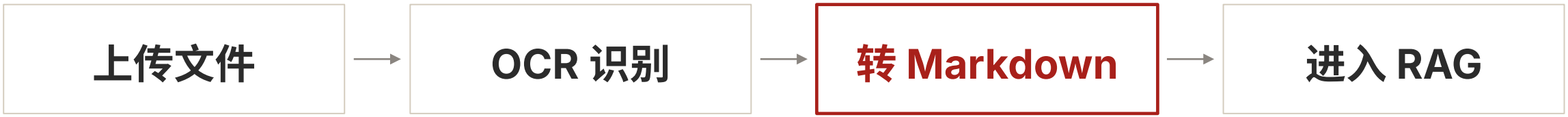
Optical Character Recognition



OCR 不只是识别字，还要尽量保留段落、表格和阅读顺序。

7.23 现场实操：调用 OCR 模型处理文件

从图片 PDF 到可检索文本



实操目标：让一份原始文件变成可问答的知识库。

08

Agent 与总结

8.1	智能体：让模型不只回答，还能做事	110
8.2	智能体可以按能力和形态分类	111
8.3	智能体开发框架，是为了组织复杂流程	112
8.4	agno 框架：一个创建智能体的工程入口	113
8.5	用 agno 创建一个智能体	114
8.6	用 agno 实操 RAG：让智能体带着资料回答	115
	总结：从函数到 LLM，再到 RAG 与 Agent	116

8.1 智能体：让模型不只回答，还能做事

Agent = 大模型 + 工具 + 记忆 + 规划

8.2 智能体可以按能力和形态分类

从聊天到工作流

对话型	**工具型**	工作流型	多智能体

Agent 的重点不是名字，而是它能不能稳定调用工具完成任务。

8.3 智能体开发框架，是为了组织复杂流程

模型、工具、状态和流程

模型	工具	记忆

Agent Framework

规划	工作流	监控

框架把一次聊天，变成可重复运行的工程系统。

8.4 agno 框架：一个创建智能体的工程入口

用框架降低组织复杂度

定义模型、工具、指令，再让 Agent 执行任务

课堂上重点看它如何把 LLM、工具和 RAG 串起来。

8.5 用 agno 创建一个智能体

从最小可运行结构开始

```
agent = Agent(
  model=llm,
  tools=[search, calculator],
  instructions="按资料回答问题"
)
```

先让 Agent 能听懂指令，再逐步增加工具能力。

8.6 用 agno 实操 RAG：让智能体带着资料回答

Agent + Retriever + Knowledge Base



这就是从“会聊天”走向“能落地”的关键一步。

总结：从函数到 LLM，再到 RAG 与 Agent

AI 的落地，本质是把模型能力接到真实数据和真实工具上

